QWEEPS

Live Interactive Visual Teaching

An implementation report and a proposal for constraint-aware scene revision

Rayven Pahignalo | ORCID: 0009-0003-2325-7968
Zenodo DOI: 10.5281/zenodo.23791785
Technical report - 30 October 2026 - Version 1.0
Report license: CC BY-NC 4.0

Abstract

QWEEPS is a prototype for generating live, interactive visual explanations from a topic or supplied document. Its teaching harness plans connected reasoning beats, selects a presentation method for each beat, directs validated scene data, and reviews content and actual browser-rendered frames before publication. Learners can seek through the explanation, inspect objects, ask contextual questions, and request a focused branch. The live browser route uses SVG and a seekable GSAP timeline connected through a Hyperframes adapter; playback does not depend on a rendered video. This report describes the inspected implementation and examines recurrent layout failures, including connectors crossing artwork, annotation collisions, and revisions based only on verbal feedback. It proposes perimeter ports, relative annotation constraints, screenshot-grounded revision and selective geometry telemetry. These changes are a design proposal, not a demonstrated improvement in success rate. A reproducible evaluation protocol and archive guidance distinguish implemented behavior, engineering tests, development observations and unresolved limitations.

Scope and evidence. This is a software and design report, not a controlled model benchmark or a study of learning outcomes. The proposed layout changes were not implemented or experimentally evaluated for this report.

Keywords: interactive explanation; visual teaching; generative AI; SVG; scene constraints; multimodal revision; human-AI interaction

Source snapshot: acceptance27/qweeps, commit 8b90989bc8855c32dde3cf545c67a9c6a3ffe9d1. The accompanying ZIP must separately identify its exact contents and version. [1]

01 / TEACHING DESIGN

Choose the method for the inference

A visually polished slide is not necessarily a useful explanation. QWEEPS treats a beat as a step that answers a learner question through an observable change. Iterations, decorative motion and paraphrases do not automatically create new beats. A shared example and stable visual identities connect the sequence, while each beat may use a different presentation method.

Learner question	Suitable visual operation	What the viewer should infer
Why does a mathematical relation hold?	Construct geometry, connect quantities and transform matching terms.	The result follows from the construction, rather than from an unexplained formula.
Who acted, where and why?	Use sourced maps, named roles, short descriptions and ordered changes.	Actors, places and consequences can be identified without prior historical familiarity.
How does a program or mechanism work?	Track an input through state, memory, signal flow or a changing part.	A particular transition produces a particular consequence.
How do choices differ?	Align concrete cases on an explicit comparison criterion.	The relevant distinction remains visible when narration moves on.

A per-beat teaching contract

New live plans declare **method**, **why**, **visualEvidence** and **textMode**. These fields make the intended teaching operation reviewable. They do not prove the operation was executed. Labels mode reserves space for geometry and keeps supporting prose available through inspection; descriptive mode retains useful definitions and explanations on the stage. [1]

Illustrative beat, not a reported generated result.
Question: why can two opposing photons have a nonzero system invariant mass?
Operation: show equal, opposite momentum arrows cancel while two energy contributions remain and add.
Evidence: the momentum resultant reaches zero while the energy total stays positive.
Misreading to prevent: the photons have acquired individual rest mass.

The next beat must address a question left by the previous result. A spoken bridge should finish connecting those ideas before the visual handoff. Neither one continuous audio recording nor a repeated layout establishes logical continuity by itself.

Manim examples and 3Blue1Brown work are visual-authoring references, not evidence that QWEEPS matches their quality. Infographics may require substantial descriptive text; geometric reasoning may require very little. Optional mnemonic devices are design choices here, with no asserted retention gain. [6,7]

02 / IMPLEMENTED ARCHITECTURE

A live scene, with review gates

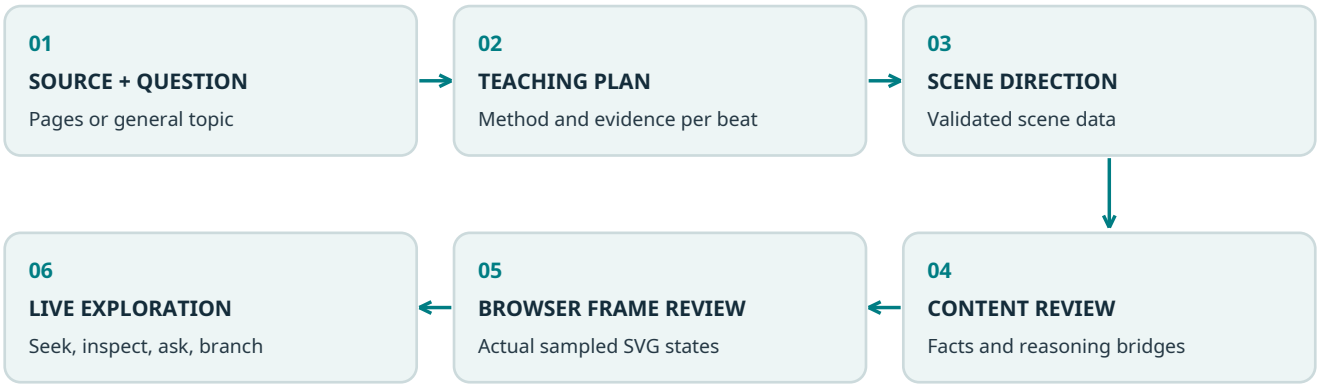


Figure 1. Conceptual view of the current pipeline. Rejected content or frames return to bounded correction steps; speech preparation follows visual approval when configured. The diagram simplifies scheduling and caches.

The browser lesson store accepts a topic, source pages and optional branch context. It plans the lesson, directs supported scene data, performs factual and narrative checks, and asks the browser to capture review frames. Publication requires an approved candidate. The source distinguishes model factual review without retrieved sources from document-grounded review. [1]

Responsibility	Inspected implementation
Teaching and validation	Python lesson store, maintained teaching skill, field constraints, targeted corrections and input-scoped checkpoints.
Rendering and time	React/SVG objects, GSAP timeline, Hyperframes frame adapter and reconstruction from an absolute playhead. Captured review images are not the playback medium.
Review and narration	Actual early and finished browser states, plus a middle state for diagrams; image-capable review; continuous narration and locally measured word cues when configured.
Exploration	Object inspection, selection, camera changes, question context and a saved parent checkpoint for branch return. Choice interactions are bounded explanatory cases, not a general numerical simulator.

Runtime boundaries

The inspected live route is a browser implementation, not an embedded Manim runtime. Native Manim-related paths also exist in the repository. Hyperframes does not choose the teaching method or automatically resolve scene geometry; the QWEEPS schema, renderer and harness retain those responsibilities. GSAP supplies seekable timeline operations. [1,5]

Lesson generation, content review, narration writing and voice synthesis are separate responsibilities. Provider/model route strings describe configuration, not independently verified model architecture or a quality ranking. Access to the relevant remote services and a local alignment runtime is needed to reproduce the full generated narration path.

03 / EVIDENCE AUDIT

What is known, and what remains open

Observation	Evidence and interpretation
Connector crossings	The diagram renderer computes center-to-center endpoints. This is a renderer policy, not solely a model coordinate error. Opaque masking is not a general solution for open constructions. [1]
Annotation collisions	Diagram groups use positioned geometry and internal SVG primitives. Independent placement can leave labels, brackets and formulas competing for the same area. No general relative-annotation solver was found in the inspected path. [1]
Revision feedback gap	The reviewer receives frame images. The director receives findings and prior scene data through a structured text request, without the failed frame image attached to that revision request. [1]
Limited geometry telemetry	The browser reports designated text-fit/overflow flags. This is useful, but is not a general pairwise visible-object collision detector. [1]
Intermittent 300-second error	A saved traceback records a request deadline during visual revision. It does not establish overthinking, overload or quota exhaustion. Temporary CLI diagnostics are cleaned up after the request. [1]

Engineering checks performed for this report

At the inspected source snapshot, **51 backend tests passed** across the browser lesson and Antigravity transport suites, and **42 frontend tests passed** across the configured studio test command. These checks exercise validation, review gates, revision caching, navigation, cue handling and other software behavior. They include controlled fixtures and mocks; they are not 93 independent teaching trials.

```
python -m pytest tests/test_visual_browser_lessons.py tests/test_antigravity_transport.py -q
# In visual-studio/: npm test
```

Evidence boundary. Development examples and user feedback motivate the proposal, but no randomized learner evaluation, controlled model comparison, first-pass success rate, cost estimate or timing distribution is reported. A passing schema or test suite does not establish that a fresh generated lesson teaches correctly.

The repository documents unequal output-budget handling between the Antigravity CLI route and strict API-backed routes. Earlier observations across those routes therefore cannot isolate model capability from adapter limits, scene complexity or revision history. [1]

04 / PROPOSED CHANGES

Move layout mechanics into the harness

Prompt guidance remains useful, but predictable geometric rules should be represented in the scene contract and enforced by code. The four interventions below are proposed work. None is claimed as an evaluated improvement in this report.

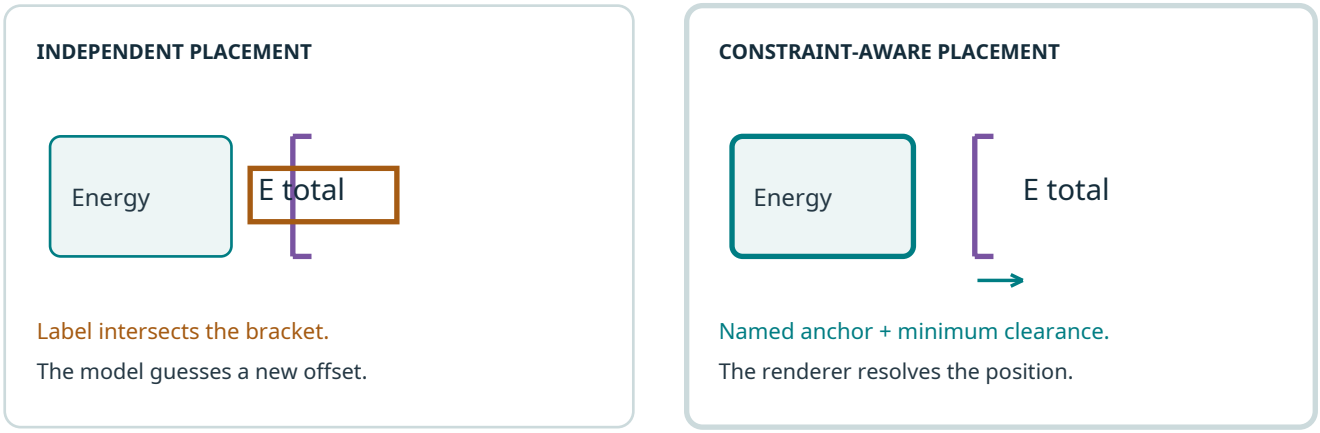


Figure 2. Schematic annotation example, drawn for this report. Left: independent placement causes a collision. Right: a named relationship and minimum clearance give the renderer a solvable placement rule. This is not a before-and-after experimental result.

1. Attach connectors to perimeter ports

Represent source and target ports explicitly, or choose them from relative direction. Account for arrowhead clearance and nearby labels. Route around unrelated objects where feasible. Draw behind cards when appropriate, but do not cover scientific artwork with opaque panels merely to conceal an incorrect path.

2. Use relative constraints for annotations

Support relationships such as rightOf, below, alignment, minimum gap and a containing region. A layout pass resolves these into coordinates. Keep explicit coordinates for data-driven positions, maps, plots, geometric constructions and physical motion: a blanket coordinate ban would damage their meaning.

3. Ground revisions in the failed image

Attach the relevant full frame and, when helpful, a focused crop to the revision request. Include frame time, stable object identifiers, current scene data and measured bounds. Preserve reviewed facts, shared metadata and unflagged beats. Add a multimodal revision path rather than inserting image paths into a text-only prompt.

4. Measure selective collisions before image review

Collect visible geometry at review sample times. Report text obstruction, inadequate clearance, unintended crossings and clipping. Use the image reviewer for semantic interpretation and overall legibility, while deterministic checks handle measurable constraints.

05 / TECHNICAL PROPOSAL

A revision packet with spatial evidence

Use stable semantic IDs for nodes and internal artwork. Every report must identify the beat, animation time, coordinate space and scene fingerprint, so a revision cannot apply stale measurements to a changed candidate. The example below is a proposed packet, not the current API schema.

```
frame: {beat: "energy-accounting", timeSeconds: 12.4}
coordinateSpace: "stage-user-units"
objects:
  formula: {x: 410, y: 210, width: 110, height: 28}
  bracket: {x: 395, y: 195, width: 35, height: 60}
finding:
  pair: ["formula", "bracket"]
  kind: "unintended-label-obstruction"
  overlap: {x: 20, y: 28}
  proposedConstraint:
    {target: "formula", rightOf: "bracket", minimumGap: 24}
```

Keep measurements in one coordinate system

Viewport pixel measurements and SVG user-space measurements are not interchangeable. getBoundingClientRect() gives viewport-relative bounds; getBBox() uses SVG-local geometry and does not incorporate ancestor transformations in the same way. Transform all reported geometry into a declared common space. Include stroke and marker clearance where supported, or use a tested conservative expansion. [2-4]

Classify intersections before treating them as defects

A label intentionally contained in a card is not a collision. Intersections within a composite icon may be necessary. A diagonal or curved path can have a large bounding rectangle despite occupying little of it. Use bounding rectangles as a broad-phase filter, then apply pair-specific tests for text, connectors, paths and containment. Record declared intentional overlaps and flag unexplained obstruction.

Solve constraints, then validate the result

Measure text after fonts load. Resolve annotation relationships within the available region, including reserved headers, conclusions and camera framing. Detect contradictory constraints or dependency cycles and return a concrete unsatisfiable-layout finding. Recompute connectors after placement; sample intermediate motion states so a clear final frame cannot hide a crossing during animation.

Bounded repair. Prefer a small layout patch when geometry alone is wrong. Re-render and remeasure it, then repeat semantic and factual checks for affected material. A numerically clear layout can still teach the wrong relationship.

06 / EVALUATION PLAN

Test the interventions separately

The main hypothesis is that deterministic constraints and image-grounded revision reduce avoidable spatial errors and repeated model calls. It remains a hypothesis. Compare changes on the same prompts and source material, without manually correcting the generated outputs.

Condition	Intervention added to the baseline
A / Baseline	Current renderer, scene contract and text-based revision feedback.
B / Ports	Perimeter endpoint and connector-clearance rules.
C / Constraints	Relative annotation layout in addition to ports.
D / Spatial telemetry	Measured, classified collisions provided to the reviser.
E / Multimodal revision	Relevant failed images plus the same scene and measurement packet.

Task set and controls

Use geometric derivations, annotated mechanisms, descriptive history, algorithm execution and comparison diagrams. Include both source-document tasks and clearly labeled general-topic tasks. Freeze the source snapshot, prompts, provider route, model ID, response limits, retry limits and fonts. Repeat trials and randomize execution order. If a provider cannot enforce an equivalent budget, report that difference instead of claiming a controlled model ranking.

Outcomes to record

Spatial quality: first-pass rejection rate; unresolved severe obstruction; text clipping; connector-to-label crossings; and false-positive collision reports.
Revision behavior: correction success by attempt, regressions to previously correct beats and preservation of semantic geometry.
Operations: stage latency, time to first response event, timeout rate, request count and actual provider-reported token/cost usage where available.
Teaching quality: blinded human judgments of method fit, factual correctness, continuity and whether the visible change supports the intended inference.

Learning claims require a separate study

If claiming improved understanding or recall, evaluate learners with consent, defined tasks and appropriate comparison conditions. Measure transfer and retention rather than relying only on preference ratings. Automated visual approval is an engineering gate, not a substitute for that study.

Report denominators, failed requests and unapproved candidates. Do not omit timeouts or treat successful retries as first-pass successes. Prespecify trial counts and analysis before collecting comparative results.

07 / REPRODUCIBILITY

Pair the report with a versioned archive

The inspected implementation is in the QWEEPS GitHub repository (acceptance27/qweeps). This report identifies its source commit but does not certify an unspecified ZIP. The deposit should identify whether its archive is that snapshot, a later implementation, a runnable bundle or a collection of reference examples. These are materially different artifacts. [1]

Archive component	Purpose
README and manifest	Exact commit, archive version, operating environment, run instructions and checksums for deposited files.
Python source and metadata	src/, pyproject.toml and the relevant tests; include the maintained teaching skill with its references.
Browser studio	visual-studio/src/, package metadata, package-lock.json, configuration and required local assets or asset-fetch instructions.
Evidence, if included	Sanitized task inputs, candidate scene data, review findings and sampled frames. Label authored references, generated trials and test fixtures separately.
Dependency and asset notes	Document remote service requirements, provider configuration, local alignment model requirements and the origin/terms of reused assets.

Minimal local verification route

```
python -m venv .venv
.venv/bin/python -m pip install -e ".[dev]"
# In visual-studio/:
npm ci
npm run build
# From repository root, with providers configured:
.venv/bin/python -m assistant.visual.server --port 4174
```

These commands outline the source workflow; they are not a claim of a clean-machine installation test. Consult the repository instructions for provider authentication, voice configuration, cached alignment models and optional native dependencies. A fresh generation still depends on external services; the code ZIP alone does not make those services reproducible.

Deposit boundaries

Public archives must omit credentials and private conversations and preserve asset attribution. QWEEPS code in this distribution uses PolyForm Noncommercial License 1.0.0; this report and QWEEPS-authored documentation use CC BY-NC 4.0. Third-party terms and permissions granted under earlier MIT releases remain unaffected.

For a combined PDF/software deposit, choose the resource type describing the main contribution, or use separate linked records when the software and report are independently significant. Zenodo documents both approaches. [10]

08 / REFERENCES AND DEPOSIT NOTES

Sources, attribution and identification

[1] QWEEPS source snapshot. acceptance27/qweeps. Commit 8b90989bc8855c32dde3cf545c67a9c6a3ffe9d1. Primary inspected paths: browser_lessons.py, llm.py, synchronization.py, alignment.py, GeneratedStage.tsx, LivePreparation.tsx and the maintained teaching skill.

[2] W3C. Scalable Vector Graphics (SVG) 2: coordinate systems, transformations and bounding boxes.

[3] MDN Web Docs. SVGGraphicsElement: getBBox() method.

[4] MDN Web Docs. Element: getBoundingClientRect() method.

[5] GSAP documentation. Timeline.

[6] Manim Community documentation. Example Gallery. Visual-authoring reference, not evidence of QWEEPS quality.

[7] 3Blue1Brown. videos repository. Visual-authoring reference.

[8] Zenodo Help. Creators. Guidance for creator names and ORCID identifiers.

[9] Zenodo Help. Digital Object Identifier (DOI). Guidance for reserving a DOI before publication.

[10] Zenodo Help. Resource types. Guidance for mixed report/software deposits.

External documentation consulted 30 October 2026. Source snapshot references identify inspected code, not a public benchmark dataset. No third-party figure or screenshot is reproduced in this report.

Author and record

Creator: Rayven Pahignalo. **ORCID:** 0009-0003-2325-7968.
Zenodo DOI: 10.5281/zenodo.23791785. Creator and record identifiers supplied by the author. [8,9]

AI assistance

Codex assisted with QWEEPS software development and preparation of this report, including source inspection, technical analysis, drafting, diagrams and PDF production. Rayven Pahignalo directed the project.

Licensing

Copyright (c) 2026 Rayven Pahignalo. This report is licensed under Creative Commons Attribution-NonCommercial 4.0 International. QWEEPS application code is offered under PolyForm Noncommercial License 1.0.0, subject to its permitted purposes. Third-party materials retain their terms. These distribution notices were updated after the cited source snapshot and do not revoke earlier MIT grants or restrict underlying ideas. See LICENSING.md and the complete license texts in the archive.